

컴퓨터 비전

Computer Vision

- 2D Image Processing 2-

동아대학교 소프트웨어대학 AI학과
2026년 1학기

임한신

RGB -> YUV 컬러 모델 변환 실습

```
import urllib.request

from google.colab.patches import cv2_imshow

# 샘플 이미지 다운로드
url =
'https://raw.githubusercontent.com/opencv/opencv/master/samples/data/smarties.png'
urllib.request.urlretrieve(url, 'smarties.png')

# 이미지 읽기
img = cv2.imread('smarties.png')

# BGR에서 YUV로 컬러 공간 변환
yuv_img = cv2.cvtColor(img, cv2.COLOR_BGR2YUV)

# 채널별로 분리 (Y, U, V)
y_channel, u_channel, v_channel = cv2.split(yuv_img)
```

```
# 배열을 가로로 결합
combined_channels = np.hstack((y_channel, u_channel, v_channel))

print("--- [원본 이미지] ---")
cv2_imshow(img)

print("\n--- [YUV 채널 분리 결과: Y(밝기) | U(청색편차) | V(적색편차)] ---")
cv2_imshow(combined_channels)
```

히스토그램 평활화 실습

```
import urllib.request

# 샘플 영상 다운로드
url = 'https://raw.githubusercontent.com/opencv/opencv/master/samples/data/vtest.avi'
urllib.request.urlretrieve(url, 'vtest.avi')

# 영상 읽기 설정
cap = cv2.VideoCapture('vtest.avi')

# 영상 정보 가져오기 (가로, 세로, FPS)
width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
fps = cap.get(cv2.CAP_PROP_FPS)
fourcc = cv2.VideoWriter_fourcc(*'mp4v') # 저장 코덱 설정

# 3. 영상 쓰기(저장) 설정
out = cv2.VideoWriter('vtest_equalized.mp4', fourcc, fps, (width, height))
```

```
while cap.isOpened():
    ret, frame = cap.read()
    if not ret:
        break

# YUV 변환 -> Y 평활화 -> 결합 -> BGR 복구
yuv = cv2.cvtColor(frame, cv2.COLOR_BGR2YUV)
y, u, v = cv2.split(yuv)

y_equalized = cv2.equalizeHist(y) # 평활화 적용

yuv_equalized = cv2.merge([y_equalized, u, v])
result_frame = cv2.cvtColor(yuv_equalized, cv2.COLOR_YUV2BGR)

# 처리된 프레임을 파일에 쓰기
out.write(result_frame)

cap.release()
out.release()

print("vtest_equalized.mp4")
```

Convolution (Filtering) 실습

```
import urllib.request

from google.colab.patches import cv2_imshow

# 샘플 이미지 다운로드

url =
'https://raw.githubusercontent.com/opencv/opencv/master/samples/data/baboon.jpg'
urllib.request.urlretrieve(url, 'baboon.jpg')
img = cv2.imread('baboon.jpg')

# 다양한 필터(커널) 정의

# 엠보싱(Embossing)
Emboss_kernel = np.array([[[-1, -1, 0],
                             [-1, 0, 1],
                             [ 0, 1, 1]]])

# 샤프닝(Sharpening)
sharp_kernel = np.array([[ 0, -1, 0],
                           [-1, 5, -1],
                           [ 0, -1, 0]])
```

```
# 평균 (Average Blur): 5x5 크기로 평균
blur_kernel = np.ones((5, 5), np.float32) / 25

# 필터 적용 (cv2.filter2D)
emboss = cv2.filter2D(img, -1, emboss_kernel)
sharp = cv2.filter2D(img, -1, sharp_kernel)
blur = cv2.filter2D(img, -1, blur_kernel)

# 결과 출력
print("--- [1. 원 본] ---")
cv2_imshow(img)
print("\n--- [2. 엠보싱 필터] ---")
cv2_imshow(cv2.add(emboss, 128))
print("\n--- [3. 샤프닝 필터] ---")
cv2_imshow(sharp)
print("\n--- [4. 평균 필터] ---")
cv2_imshow(blur)
```

Convolution (Filtering) 실습 (Gaussian)

```
import urllib.request

from google.colab.patches import cv2_imshow

# 샘플 이미지 다운로드

url =
'https://raw.githubusercontent.com/opencv/opencv/master/samples/data/baboon.jpg'
urllib.request.urlretrieve(url, 'baboon.jpg')
img = cv2.imread('baboon.jpg')

# 가우시안 필터 적용 (cv2.GaussianBlur)
# ksize: 커널 크기 (홀수), sigmaX: X방향 표준편차
blur_3x3 = cv2.GaussianBlur(img, (3, 3), 0)
blur_9x9 = cv2.GaussianBlur(img, (9, 9), 0)

# 직접 커널을 정의해서 적용하기
# 5x5 가우시안 커널 생성
kernel_5x5 = cv2.getGaussianKernel(5, 0)
# 1차원 커널을 외적하여 2차원 행렬로 변환
gaussian_kernel = np.outer(kernel_5x5, kernel_5x5)
custom_blur = cv2.filter2D(img, -1, gaussian_kernel)
```

```
# 결과 출력
print("--- [1. 원 본] ---")
cv2_imshow(img)
print("\n--- [2. 3x3 가우시안] ---")
cv2_imshow(blur_3x3)
print("\n--- [3. 9x9 가우시안] ---")
cv2_imshow(blur_9x9)
print("\n--- [4. 직접 정의한 5x5 커널] ---")
cv2_imshow(custom_blur)
```

Interpolation (보간) 실습

```
import urllib.request

from google.colab.patches import cv2_imshow

# 샘플 이미지 다운로드
url = 'https://raw.githubusercontent.com/opencv/opencv/master/samples/data/messi5.jpg'
urllib.request.urlretrieve(url, 'messi.jpg')
img = cv2.imread('messi.jpg')

# 테스트를 위해 이미지를 작게 축소 (100x100)
small_img = cv2.resize(img, (100, 100), interpolation=cv2.INTER_AREA)

# 5배 확대 (500x500)
# (1) Nearest neighbor
nearest = cv2.resize(small_img, (500, 500), interpolation=cv2.INTER_NEAREST)
# (2) Bilinear
linear = cv2.resize(small_img, (500, 500), interpolation=cv2.INTER_LINEAR)
# (3) Bicubic
cubic = cv2.resize(small_img, (500, 500), interpolation=cv2.INTER_CUBIC)
```

```
# 결과 출력
print("--- [1. 축소된 원본 (100x100)] ---")
cv2_imshow(small_img)
print("\n--- [2. Nearest neighbor] ---")
cv2_imshow(nearest)
print("\n--- [3. Bilinear] ---")
cv2_imshow(linear)
print("\n--- [4. Bicubic] ---")
cv2_imshow(cubic)
```

감마 조절(gamma correction) 실습

```
import urllib.request

from google.colab.patches import cv2_imshow

def adjust_gamma(image, gamma=1.0):
    # 0~255 범위의 룩업 테이블(LUT) 생성
    inv_gamma = 1.0 / gamma
    table = np.array([((i / 255.0) ** inv_gamma) * 255
                      for i in np.arange(0, 256)]).astype("uint8")

    return cv2.LUT(image, table)

# 샘플 이미지 다운로드
url = 'https://raw.githubusercontent.com/opencv/opencv/master/samples/data/fruits.jpg'
urllib.request.urlretrieve(url, 'fruits.jpg')
img = cv2.imread('fruits.jpg')

# 감마 값 적용
bright_gamma = adjust_gamma(img, gamma=2.2)
dark_gamma = adjust_gamma(img, gamma=0.4)
```

```
# 결과 출력 (가로 병합)
res = np.hstack((img, bright_gamma, dark_gamma))

print("--- [순서: 원본 | 감마 2.2 | 감마 0.4] ---")
cv2_imshow(res)
```

모폴로지(Morphology) 실습

```
import urllib.request
from google.colab.patches import cv2_imshow

# 샘플 이미지 다운로드
url =
'https://raw.githubusercontent.com/opencv/opencv/master/samples/data/smarties.png'
urllib.request.urlretrieve(url, 'smarties.png')
img = cv2.imread('smarties.png')

# 흑백 이진화(Thresholding) 수행
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
_, binary = cv2.threshold(gray, 0, 255, cv2.THRESH_BINARY_INV +
cv2.THRESH_OTSU)

# 구조 요소(Kernel) 생성 (5x5 크기)
kernel = np.ones((5, 5), np.uint8)

# 모폴로지 연산 적용
erosion = cv2.erode(binary, kernel, iterations=1) # 침식
dilation = cv2.dilate(binary, kernel, iterations=1) # 팽창
```

```
opening = cv2.morphologyEx(binary, cv2.MORPH_OPEN, kernel) # 열림
closing = cv2.morphologyEx(binary, cv2.MORPH_CLOSE, kernel) # 닫힘

# 결과 출력 (가로로 병합하여 비교)
res1 = np.hstack((binary, erosion, dilation))
res2 = np.hstack((binary, opening, closing))

print("--- [상단: 이진화 원본 | 침식 | 팽창] ---")
cv2_imshow(res1)
print("\n--- [하단: 이진화 원본 | 열림 | 닫힘] ---")
cv2_imshow(res2)
```


기하 변환 실습

```
import urllib.request

from google.colab.patches import cv2_imshow

# 샘플 이미지 다운로드 (Sudoku)

url =
'https://raw.githubusercontent.com/opencv/opencv/master/samples/data/sudoku.png'
urllib.request.urlretrieve(url, 'sudoku.png')

img = cv2.imread('sudoku.png')

rows, cols = img.shape[:2]

# 이동 변환(Translation) - 가로 100, 세로 50 이동
# 변환 행렬 M = [[1, 0, tx], [0, 1, ty]]
M_translation = np.float32([[1, 0, 100], [0, 1, 50]])
dst_translation = cv2.warpAffine(img, M_translation, (cols, rows))

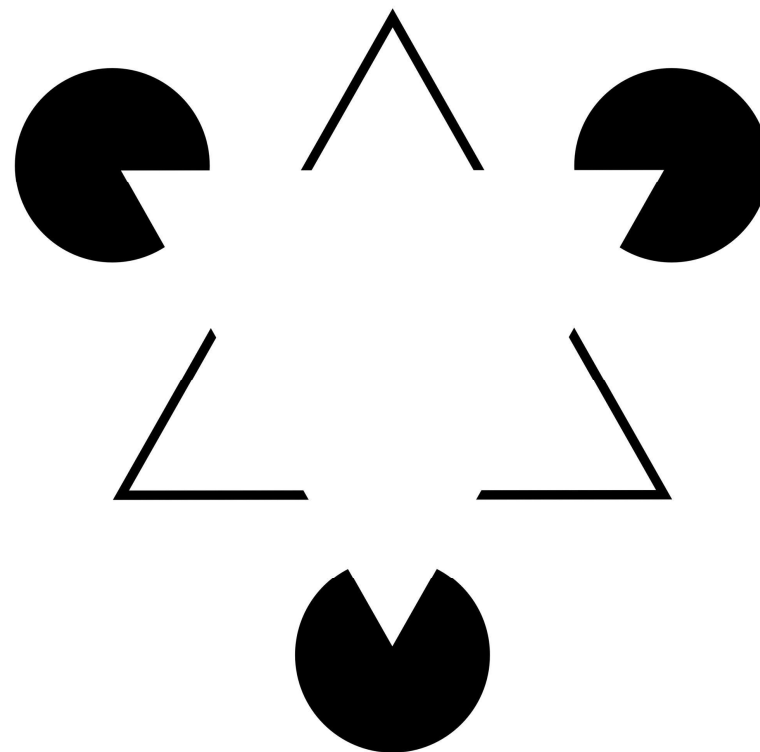
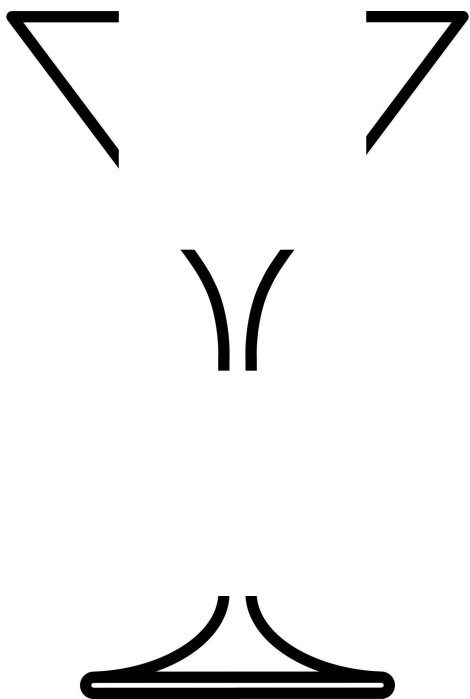
# 회전 + 크기 변환 (Rotation + Scale) - 중심축 기준 45도 회전, 0.7배 축소
# getRotationMatrix2D(중심좌표, 각도, 배율)
M_rotation = cv2.getRotationMatrix2D((cols/2, rows/2), 45, 0.7)
dst_rotation = cv2.warpAffine(img, M_rotation, (cols, rows))
```

```
# 결과 출력
print("--- [1. 원본] ---")
cv2_imshow(img)

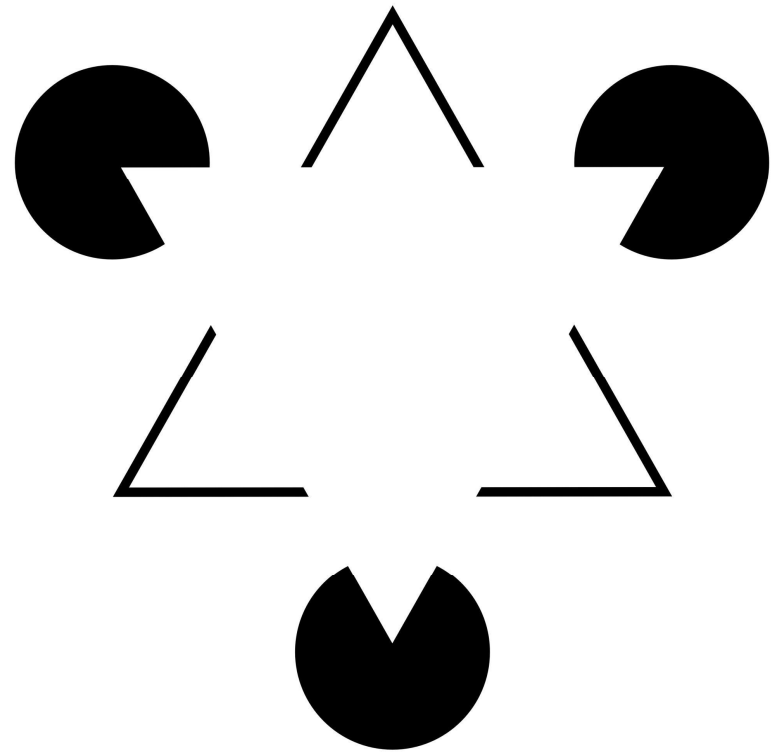
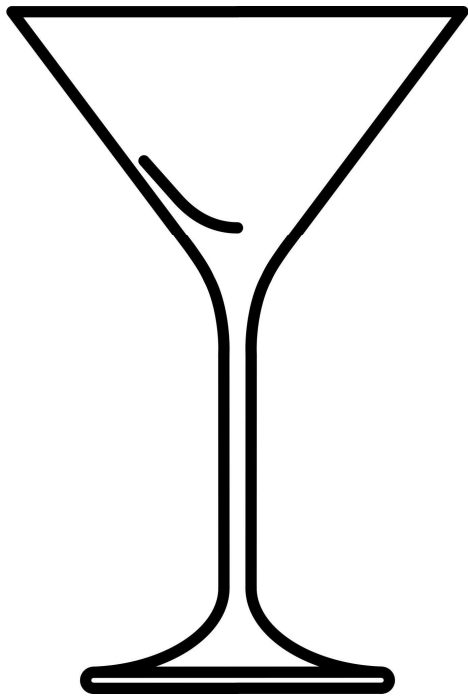
print("--- [2. 이동 변환 (Translation)] ---")
cv2_imshow(dst_translation)

print("\n--- [3. 회전 + 크기 변환 (Rotation + Scale)] ---")
cv2_imshow(dst_rotation)
```

에지 검출

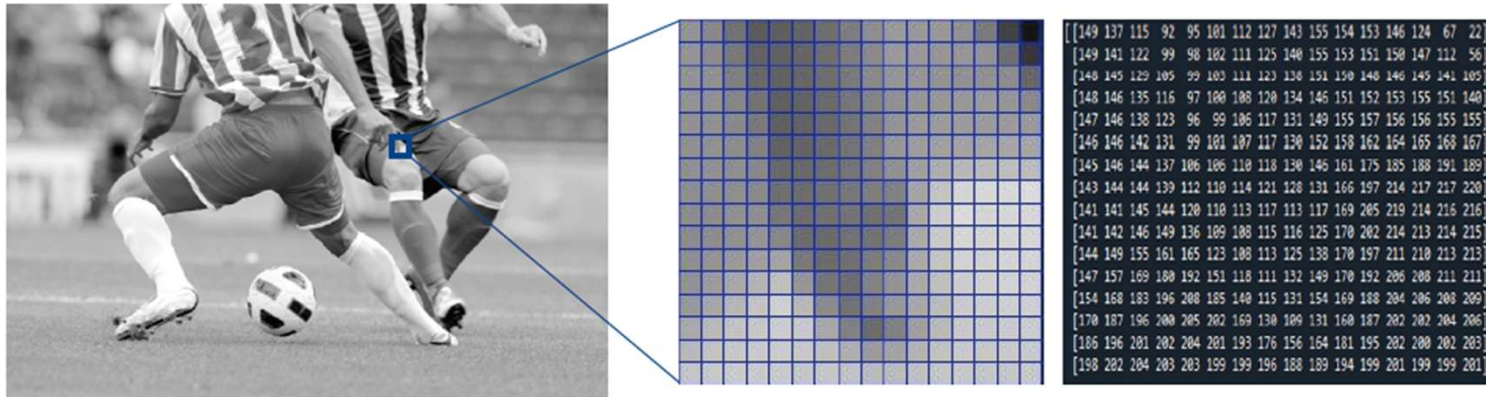


에지 검출



에지 검출

- 에지 검출 알고리즘
 - 물체 내부는 명암이 서서히 변하고 경계는 급격히 변하는 특성을 활용

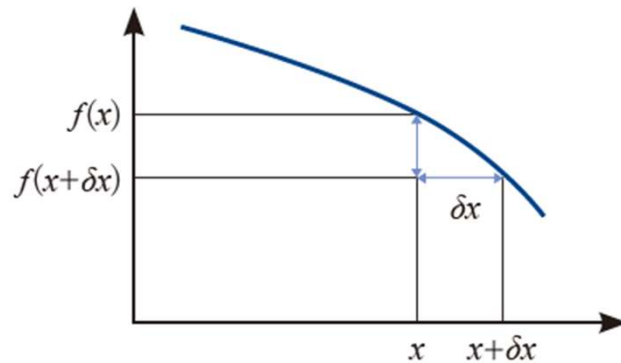


에지 검출

- 에지 검출 알고리즘
 - 미분을 디지털 영상에 적용하면

$$f'(x) = \frac{f(x+\delta x) - f(x)}{\delta x} = f(x+1) - f(x)$$

- 실제 구현은 필터 u 로 컨볼루션 (u 를 에지 연산자라 부름)



연속 함수의 미분



디지털 영상의 미분(필터 u 로 컨볼루션)

에지 검출

- 에지 검출 알고리즘
 - 1차 미분과 2차 미분

$$f''(x) = \frac{f'(x) - f'(x-\delta)}{\delta} = f'(x) - f'(x-1)$$

$$= (f(x+1) - f(x)) - (f(x) - f(x-1)) \quad \text{2차 미분}$$

$$= f(x+1) - 2f(x) + f(x-1)$$

이 식을 구현하는 필터는

1	-2	1
---	----	---

두꺼운 에지로 인해 위치찾기 문제 발생

	0	1	2	3	4	5	6	7	8	9	0	1
원래 영상 f	3	3	3	3	5	7	7	7	7	3	3	3
필터 $u = \begin{bmatrix} -1 & 1 \end{bmatrix}$ 로 컨볼루션												
1차 미분 f'	0	0	0	2	2	0	0	0	-4	0	0	-
필터 $u = \begin{bmatrix} -1 & 1 \end{bmatrix}$ 로 컨볼루션												
2차 미분 f''	0	0	2	0	-2	0	0	-4	4	0	-	-

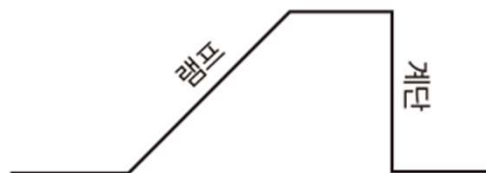
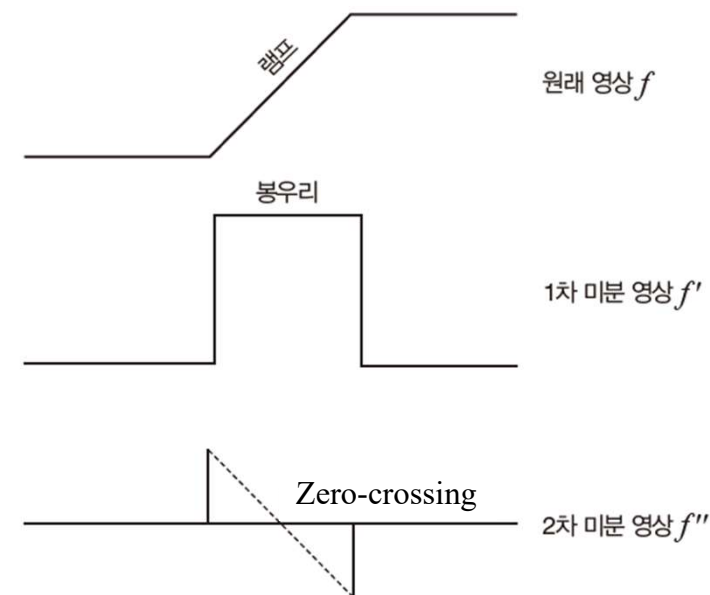


현실 세계에서 발생하는 램프 에지

에지 검출

- 에지 검출 알고리즘
 - 1차 미분과 2차 미분

두꺼운 에지로 인해 위치찾기 문제 발생



현실 세계에서 발생하는 램프 에지

에지 검출

- 에지 검출 알고리즘
 - 1차 미분에 기반한 에지 연산자
 - 크기가 2인 필터를 크기 3으로 확장

$$f'_x(y, x) = f(y, x+1) - f(y, x-1)$$

$$f'_y(y, x) = f(y+1, x) - f(y-1, x)$$

$$u_x = \begin{bmatrix} -1 & 0 & 1 \end{bmatrix} \quad u_y = \begin{bmatrix} -1 \\ 0 \\ 1 \end{bmatrix}$$

- 또한 1차원을 2차원으로 확장

$$u_x = \begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix} \quad u_y = \begin{bmatrix} -1 & -1 & -1 \\ 0 & 0 & 0 \\ 1 & 1 & 1 \end{bmatrix}$$

프레윗(Prewitt) 연산자

$$u_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix} \quad u_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

소벨(Sobel) 연산자

에지 검출

- 에지 검출 알고리즘
 - 1차 미분에 기반한 에지 연산자
 - 크기가 2인 필터를 크기 3으로 확장

$$f'_x(y, x) = f(y, x+1) - f(y, x-1)$$

$$f'_y(y, x) = f(y+1, x) - f(y-1, x)$$

$$u_x = \begin{bmatrix} -1 & 0 & 1 \end{bmatrix} \quad u_y = \begin{bmatrix} -1 \\ 0 \\ 1 \end{bmatrix}$$

- 또한 1차원을 2차원으로 확장

$$u_x = \begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix} \quad u_y = \begin{bmatrix} -1 & -1 & -1 \\ 0 & 0 & 0 \\ 1 & 1 & 1 \end{bmatrix}$$

프레윗(Prewitt) 연산자

$$\text{에지 강도 : } (y, x) = \sqrt{f'_x(y, x)^2 + f'_y(y, x)^2}$$

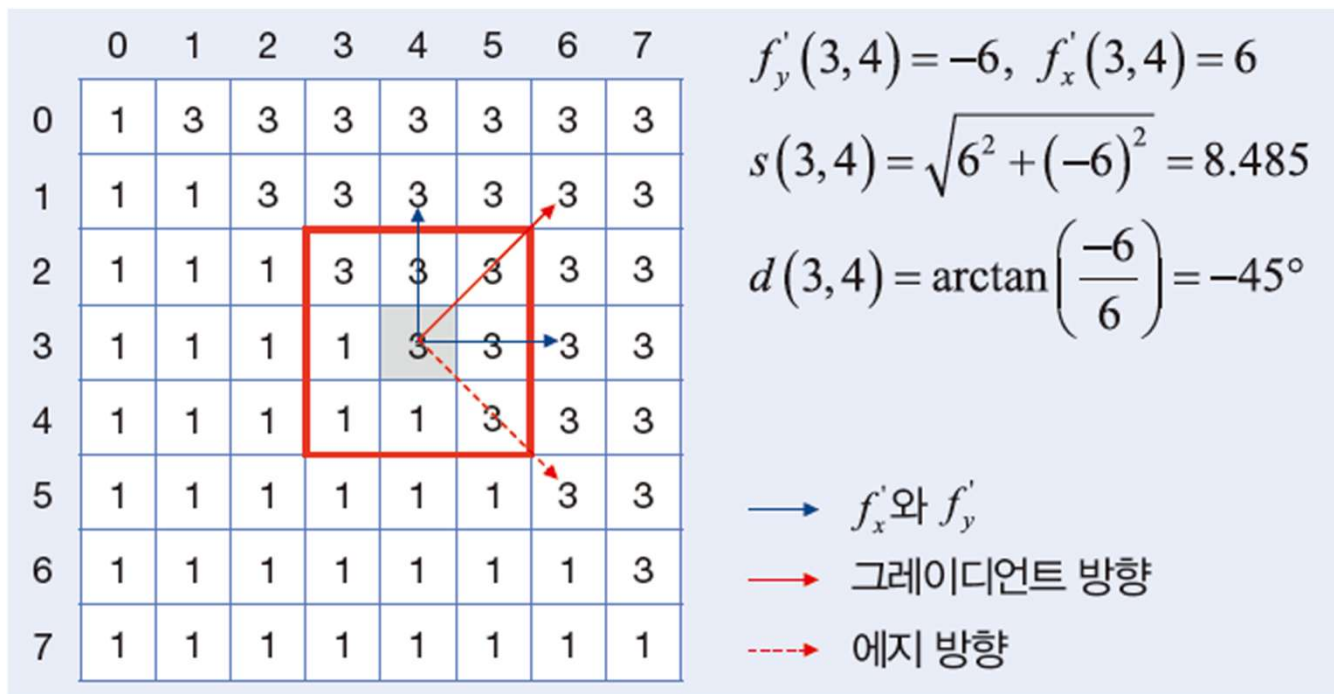
$$\text{Gradient 방향 : } (y, x) = \arctan\left(\frac{f'_y(y, x)}{f'_x(y, x)}\right)$$

$$u_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix} \quad u_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

소벨(Sobel) 연산자

에지 검출

- 에지 검출 알고리즘
 - 1차 미분에 기반한 에지 연산자



소벨 연산자 적용 예시

소벨(Sobel) 에지 검출 실습

```
import cv2
import numpy as np
import urllib.request
from google.colab.patches import cv2_imshow

# 샘플 이미지 다운로드
url =
'https://raw.githubusercontent.com/opencv/opencv/master/samples/data/baboon.jpg'
urllib.request.urlretrieve(url, 'baboon.jpg')

# 미지 읽기 및 흑백 변환
img = cv2.imread('baboon.jpg')
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# 소벨 에지 검출
# cv2.Sobel(영상, 출력 데이터 타입, x방향 미분값, y방향 미분값, 커널 크기)
# 데이터 타입은 정밀도를 위해 float64(CV_64F)를 사용한 뒤 나중에 8비트로 변환
합니다.
sobel_x = cv2.Sobel(gray, cv2.CV_64F, 1, 0, ksize=3) # 수직선 검출
sobel_y = cv2.Sobel(gray, cv2.CV_64F, 0, 1, ksize=3) # 수평선 검출
```

```
# 결과값의 절댓값을 취하고 8비트로 변환
abs_sobel_x = cv2.convertScaleAbs(sobel_x)
abs_sobel_y = cv2.convertScaleAbs(sobel_y)

# X축과 Y축 결과를 합쳐서 최종 에지 영상 생성
sobel_combined = cv2.addWeighted(abs_sobel_x, 0.5, abs_sobel_y, 0.5, 0)

# 결과 출력
print("--- [원본 이미지] ---")
cv2_imshow(img)
print("\n--- [Sobel X (수직 경계)] ---")
cv2_imshow(abs_sobel_x)
print("\n--- [Sobel Y (수평 경계)] ---")
cv2_imshow(abs_sobel_y)
print("\n--- [Sobel Combined (최종 에지)] ---")
cv2_imshow(sobel_combined)
```

에지 검출

- 캐니(Canny) 에지 검출 알고리즘
 - 기존 방식들의 단점
 - 노이즈에 취약하거나 에지가 너무 두껍게 표현됨
 - 단계
 1. 가우시안 필터를 적용하여 노이즈 제거
 2. 소벨(Sobel) 마스크 등을 사용해 각 픽셀에서의 밝기 변화량(크기)과 변화 방향(각도)을 구함
 3. 에지가 두껍게 나타나는 것을 방지하기 위해 그래디언트 방향에서 가장 값이 큰(극대점인) 픽셀만 남기고 나머지는 제거
 4. 이중 임계값(T_{high} , T_{low})으로 강한 에지(T_{high} 보다 큼)와 약한 에지(T_{high} 와 T_{low} 의 사이)를 분류
 5. 약한 에지 중에서 강한 에지와 연결된 것만 진짜 에지로 인정하고 나머지는 제거



캐니(Canny) 에지 검출 실습

```
import cv2
import numpy as np
import urllib.request
from google.colab.patches import cv2_imshow

url = 'https://raw.githubusercontent.com/opencv/opencv/master/samples/data/fruits.jpg'
urllib.request.urlretrieve(url, 'fruits.jpg')

# 이미지 읽기
img = cv2.imread('fruits.jpg')

# 에지 검출 성능을 높이기 위해 흑백(Grayscale) 변환
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

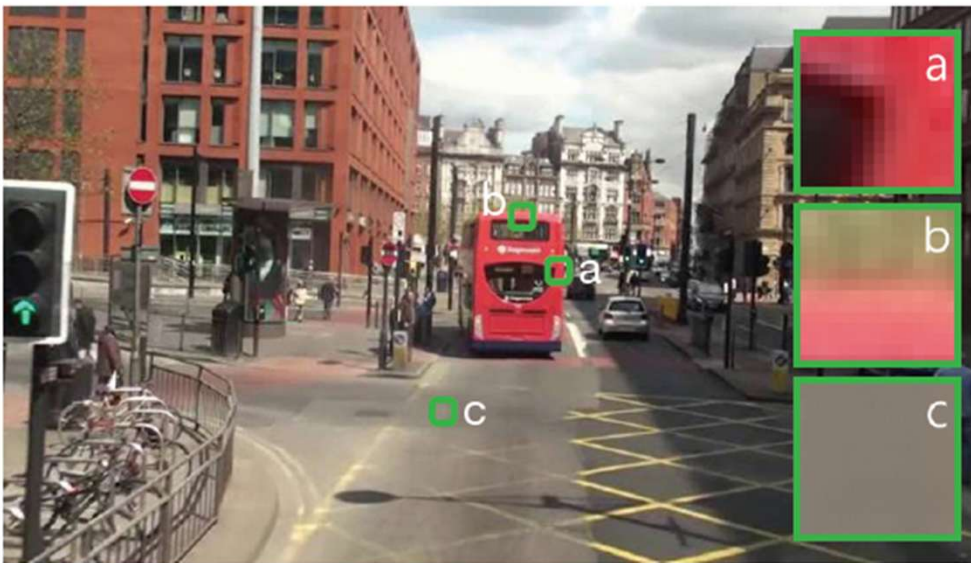
# 가우시안 블러로 노이즈 제거
blur = cv2.GaussianBlur(gray, (5, 5), 0)
```

```
# 캐니 에지 적용
# cv2.Canny(이미지, 하한 임계값, 상한 임계값)
edges = cv2.Canny(blur, 100, 200)

# 결과 출력 (원본과 에지 영상)
print("--- 원본 이미지 ---")
cv2_imshow(img)
print("\n--- 캐니 에지 결과 ---")
cv2_imshow(edges)
```

코너 검출

- 코너 검출(Corner detection) 알고리즘



왼쪽 영상의 a, b, c 중 오른쪽 영상에서 가장 찾기 쉬운 것은?

코너 검출

- 해리스 코너 검출(Harris corner detection) 알고리즘

$$E(u, v) = \sum_{x, y} w(x, y) [I(x + u, y + v) - I(x, y)]^2$$

$w(x, y)$: Gaussian 필터 또는 1

(u, v) 방향의 영상의 밝기의 변화량

보통 $-1 \leq u \leq 1, -1 \leq v \leq 1$

-1, -1		
		1, 1

코너 검출

- 해리스 코너 검출(Harris corner detection) 알고리즘

$$E(u, v) = \sum_{x, y} w(x, y) [I(x + u, y + v) - I(x, y)]^2$$

(u, v) 방향의 영상의 밝기의 변화량

보통 $-1 \leq u \leq 1, -1 \leq v \leq 1$

$w(x, y)$: Gaussian 필터 또는 1

원래 영상

	0	1	2	3	4	5	6	7	8	9
0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	0	0
2	0	0	0	1	0	0	0	0	0	0
3	0	0	0	1	1	0	0	0	0	0
4	0	0	0	1	1	1	0	0	0	0
5	0	0	0	1	1	1	1	0	0	0
6	0	0	0	1	1	1	1	1	0	0
7	0	0	0	0	0	0	0	0	0	0
8	0	0	0	0	0	0	0	0	0	0
9	0	0	0	0	0	0	0	0	0	0

$E(u, v)$

	u		
	-1	0	1
-1	3	4	4
0	2	0	2
1	4	3	2

a

	u		
	-1	0	1
-1	3	1	6
0	3	0	4
1	3	0	3

b

	u		
	-1	0	1
-1	0	0	0
0	0	0	0
1	0	0	0

c

-1, -1		
		1, 1

a 가 코너로 가장 적당

코너 검출

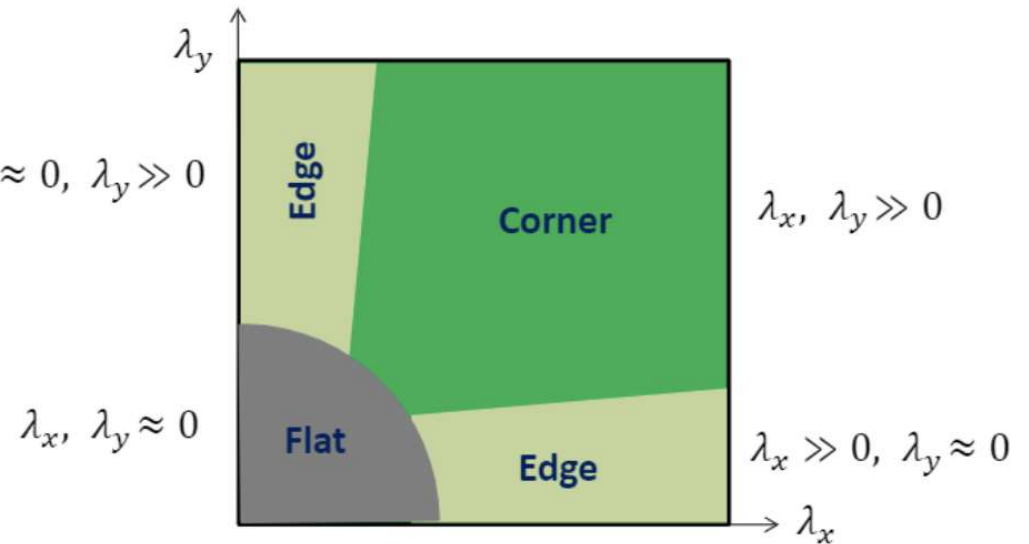
- 해리스 코너 검출(Harris corner detection) 알고리즘

$$E(u, v) = \sum_{x, y} w(x, y) [I(x + u, y + v) - I(x, y)]^2$$

$$\approx [u \quad v] M \begin{bmatrix} u \\ v \end{bmatrix}$$

where $M = \sum_{x, y} w(x, y) \begin{bmatrix} I_x I_x & I_x I_y \\ I_x I_y & I_y I_y \end{bmatrix}$ $\lambda_x \approx 0, \lambda_y \gg 0$

I_x, I_y : x, y 방향에서의 밝기 변화(Sobel 에지값)



코너 검출

- 해리스 코너 검출(Harris corner detection) 알고리즘

$$E(u, v) = \sum_{x, y} w(x, y) [I(x + u, y + v) - I(x, y)]^2$$

$$\approx [u \quad v] M \begin{bmatrix} u \\ v \end{bmatrix}$$

where $M = \sum_{x, y} w(x, y) \begin{bmatrix} I_x I_x & I_x I_y \\ I_x I_y & I_y I_y \end{bmatrix}$ $\lambda_x \approx 0, \lambda_y \gg 0$

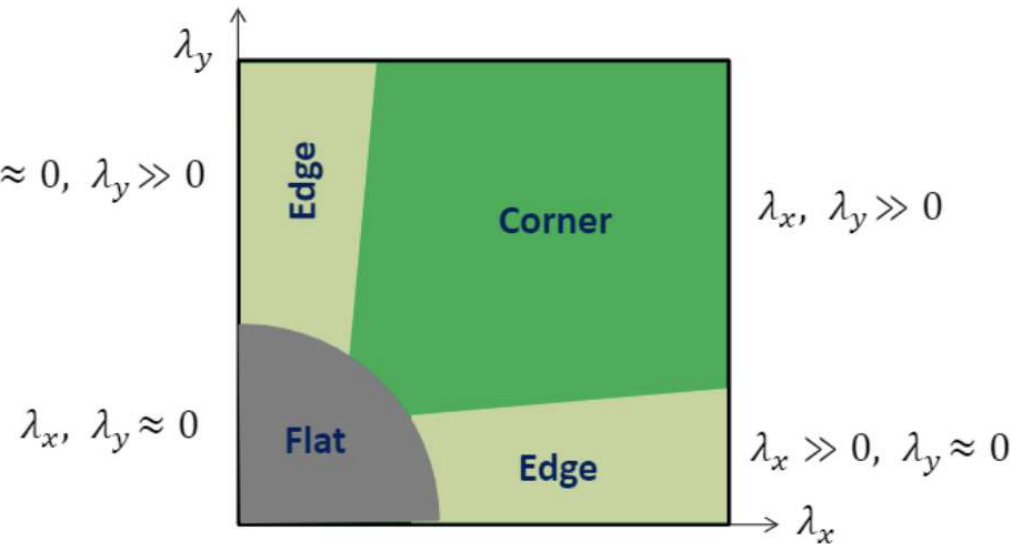
I_x, I_y : x, y 방향에서의 밝기 변화(Sobel 에지값)



Eigen value decomposition(고유값분해)



λ_x, λ_y



코너 검출

- 해리스 코너 검출(Harris corner detection) 알고리즘

$$E(u, v) = \sum_{x, y} w(x, y) [I(x + u, y + v) - I(x, y)]^2$$

$$\approx \begin{bmatrix} u & v \end{bmatrix} M \begin{bmatrix} u \\ v \end{bmatrix}$$

where $M = \sum_{x, y} w(x, y) \begin{bmatrix} I_x I_x & I_x I_y \\ I_x I_y & I_y I_y \end{bmatrix}$

I_x, I_y : x, y 방향에서의 밝기 변화(Sobel 에지값)



Eigen value decomposition(고유값분해)



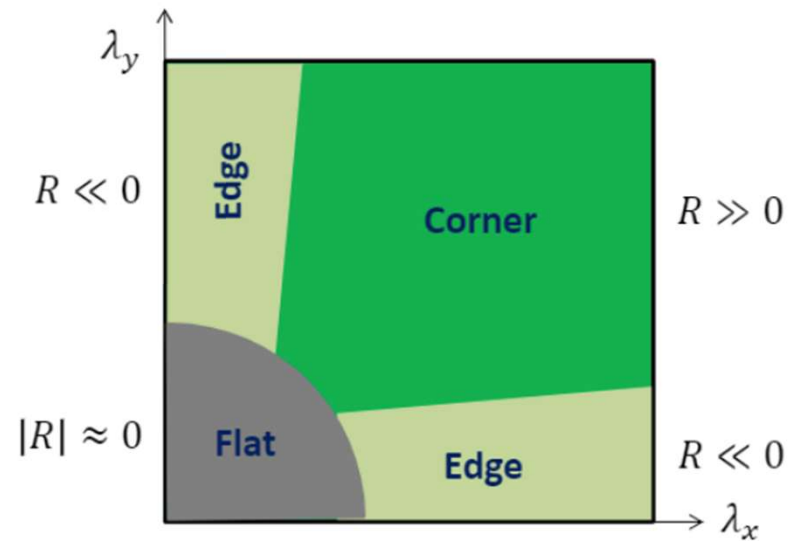
λ_x, λ_y



$$R = \det(M) - k(\text{trace}(M))^2$$

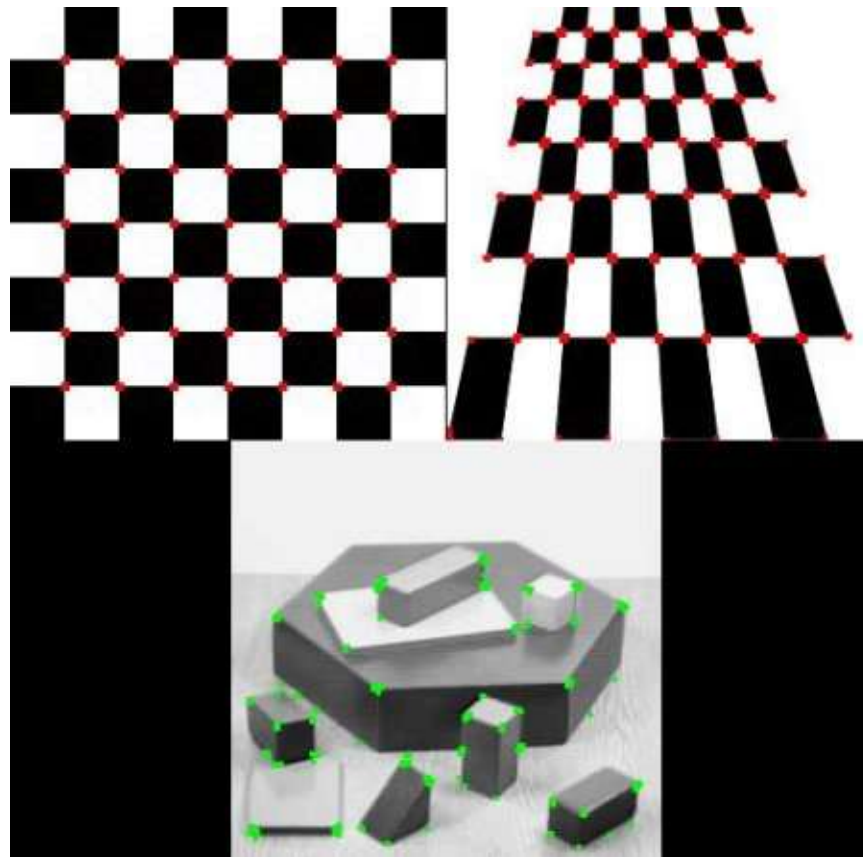
$$\begin{aligned} \det(M) &= \lambda_1 \lambda_2 \\ \text{trace}(M) &= \lambda_1 + \lambda_2 \end{aligned}$$

k : 상수



코너 검출

- 해리스 코너 검출(Harris corner detection) 알고리즘



해리스 코너 검출 실습

```
import urllib.request

from google.colab.patches import cv2_imshow

# 샘플 이미지 다운로드
url =
'https://raw.githubusercontent.com/opencv/opencv/master/samples/data/sudoku.jpg'
urllib.request.urlretrieve(url, 'sudoku.jpg')
img = cv2.imread('sudoku.jpg')

# 코너 검출은 그레이 이미지에서 수행
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
gray = np.float32(gray) # 계산 정밀도를 위해 float32로 변환

# 해리스 코너 검출 수행
# gray: 입력영상 (float32 타입)
# 2: 블록 크기 (Block Size). 코너 검출을 위한 윈도우 크기
# 3: 소벨 커널 크기 (Sobel Kernel Size). 미분 값을 계산할 때 사용하는 커널 크기
# - 0.04: Harris 코너 검출 방정식의 k값 (보통 0.04~0.06 사용)
dst = cv2.cornerHarris(gray, 2, 3, 0.04)
```

```
# 검출된 코너 점들을 약간 크고 뚜렷하게 만듦
dst = cv2.dilate(dst, None)

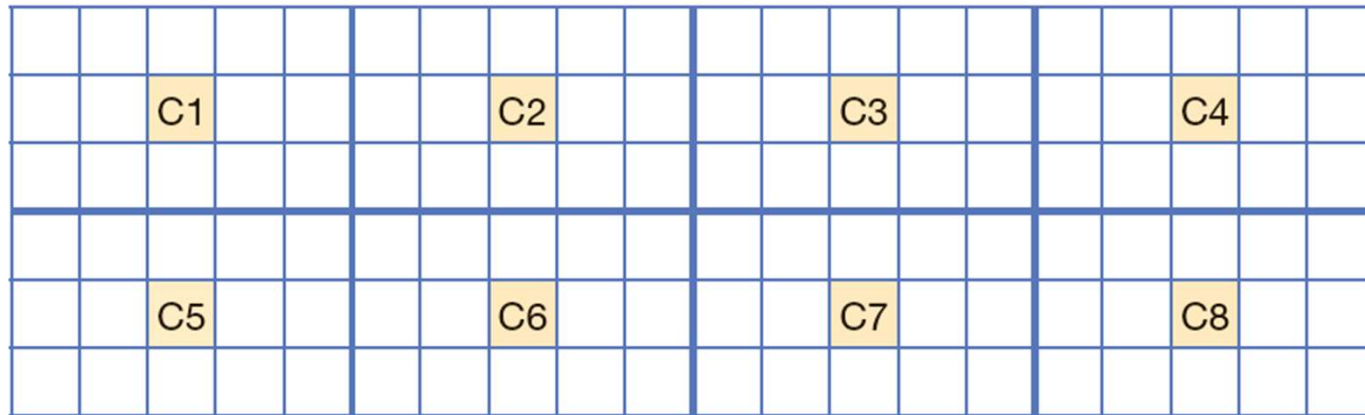
# 영상 내 가장 강한 코너 점의 값(dst.max())의 1%보다 큰 픽셀만 코너로 간주
threshold = 0.01 * dst.max()

# dst > threshold 조건을 만족하는 좌표의 픽셀을 빨간색 [0, 0, 255]로 변경
img_result = img.copy()
img_result[dst > threshold] = [0, 0, 255]

# 결과 출력
print("--- [원본 이미지] ---")
cv2_imshow(img)
print("\n--- [해리스 코너 검출 결과 (빨간색 점)] ---")
cv2_imshow(img_result)
```

슈퍼픽셀(Superpixel) 분할

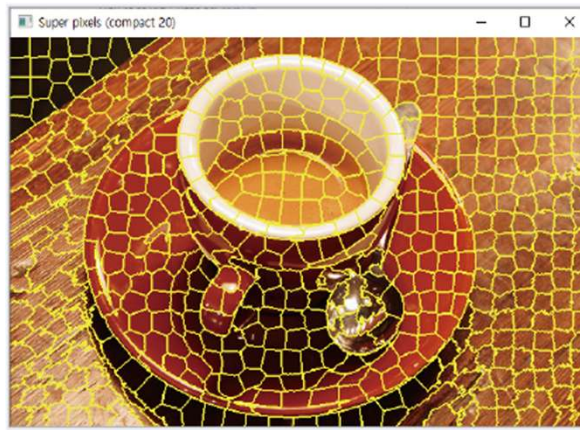
- 슈퍼픽셀(Superpixel)은 픽셀보다 크지만 물체보다 작은 자잘한 영역으로 과잉 분할(over-segmentation)된 단위
- SLIC(Simple Linear Iterative Clustering) 알고리즘
 - 각 픽셀을 (R, G, B, x, y) 의 5차원 데이터로 취급
 - k-means clustering과 비슷하게 동작 (주변 픽셀 할당 단계와 cluster들의 중심을 갱신하는 단계를 반복)
 - 파라미터
 - K : 슈퍼픽셀의 개수
 - Compactness : 얼마나 촘촘하게 뭉칠지 결정



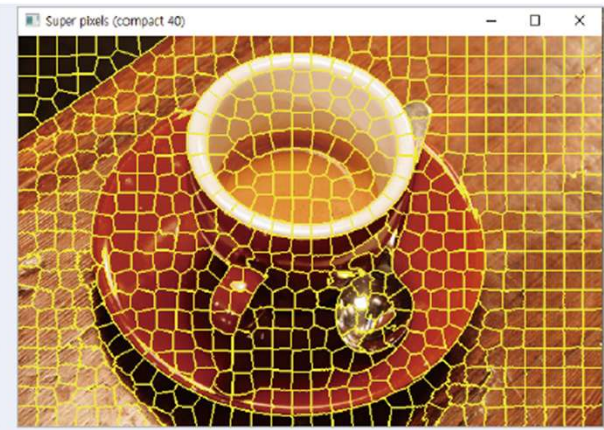
SLIC 알고리즘의 초기 cluster들의 중심

슈퍼픽셀(Superpixel) 분할

- 슈퍼픽셀(Superpixel)은 픽셀보다 크지만 물체보다 작은 자잘한 영역으로 과잉 분할(over-segmentation)된 단위
- SLIC(Simple Linear Iterative Clustering) 알고리즘
 - 각 픽셀을 (R, G, B, x, y) 의 5차원 데이터로 취급
 - k-means clustering과 비슷하게 동작 (주변 픽셀 할당 단계와 cluster들의 중심을 갱신하는 단계를 반복)
 - 파라미터
 - K : 슈퍼픽셀의 개수
 - Compactness : 얼마나 촘촘하게 뭉칠지 결정



compactness=20



compactness=40

슈퍼픽셀(Superpixel) 분할

- 슈퍼픽셀(Superpixel)은 픽셀보다 크지만 물체보다 작은 자잘한 영역으로 과잉 분할(over-segmentation)된 단위
- SLIC(Simple Linear Iterative Clustering) 알고리즘
 - 각 픽셀을 (R, G, B, x, y)의 5차원 데이터로 취급
 - k-means clustering과 비슷하게 동작 (주변 픽셀 할당 단계와 cluster들의 중심을 갱신하는 단계를 반복)
 - 파라미터
 - K : 슈퍼픽셀의 개수
 - Compactness : 얼마나 촘촘하게 뭉칠지 결정
 - 장점
 - 연산 효율성이 커짐. 예) 픽셀 수가 1,000,000개인 이미지를 500개의 슈퍼 픽셀로 줄이면 후속 알고리즘의 계산량이 크게 감소
 - 영상 내 실제 사물의 경계선을 어느 정도 보존하면서 뭉쳐줌
 - 활용
 - 주로 슈퍼 픽셀들로 그래프를 만들어서 활용(객체 분할, 깊이추정 등)

슈퍼픽셀(Superpixel) 분할 실습

```
import urllib.request

import matplotlib.pyplot as plt

from skimage.segmentation import slic, mark_boundaries
from skimage.color import label2rgb

# 샘플 이미지 다운로드

url =
'https://raw.githubusercontent.com/opencv/opencv/master/samples/data/butterfly.jpg'

urllib.request.urlretrieve(url, 'butterfly.jpg')

img = cv2.imread('butterfly.jpg')

img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

# SLIC 알고리즘 적용

segments = slic(img_rgb, n_segments=400, compactness=10, sigma=1, start_label=1)

# 평균 색상으로 채우기

superpixel_avg = label2rgb(segments, img_rgb, kind='avg')
```

```
# 결과 시각화

plt.figure(figsize=(18, 6))

plt.subplot(1, 3, 1)

plt.title("Original (Butterfly)")

plt.imshow(img_rgb)

plt.axis('off')

plt.subplot(1, 3, 2)

plt.title("SLIC Boundaries (400 segments)")

plt.imshow(mark_boundaries(img_rgb, segments))

plt.axis('off')

plt.subplot(1, 3, 3)

plt.title("Segmented (Avg Color)")

plt.imshow(superpixel_avg.astype('uint8'))

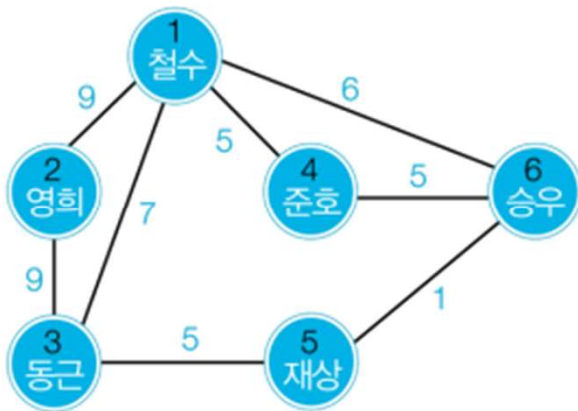
plt.axis('off')

plt.tight_layout()

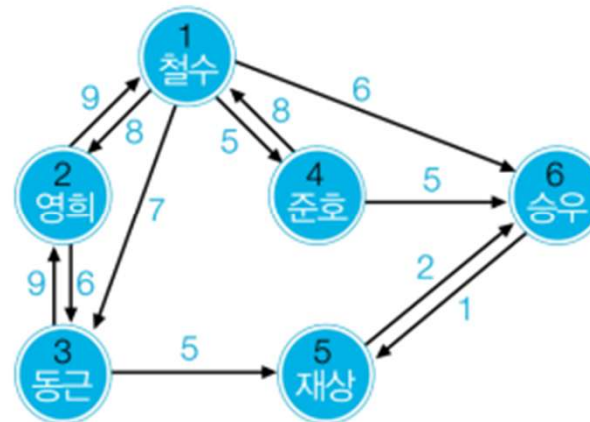
plt.show()
```

최적화 분할

- 그래프의 표현
 - 그래프는 현상이나 사물을 정점(vertex)과 간선(edge)로 표현
 - 그래프 $G=(V, E)$
 - V : n 개의 정점 집합, E : 정점 간에 존재하는 간선 집합
 - 두 정점이 간선으로 연결되어 있으면 인접(adjacent)하다고 함



가중치를 가진 무방향 그래프



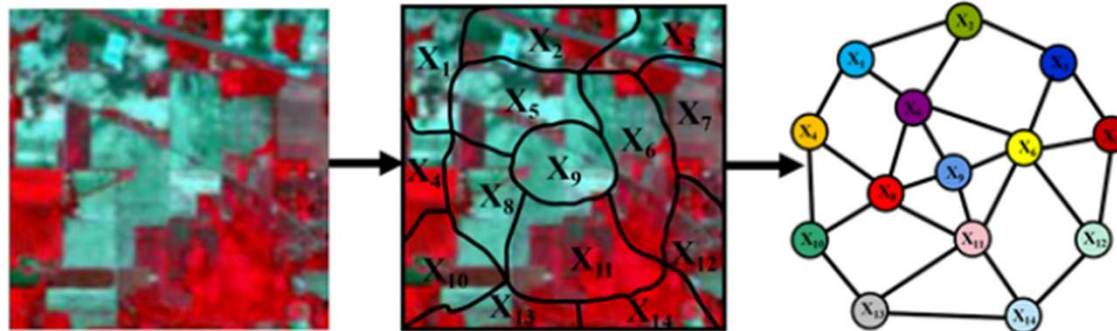
가중치를 가진 방향 그래프

최적화 분할

- 영상의 그래프 표현
 - 픽셀 또는 슈퍼픽셀을 노드(정점)로 취함
 - 두 노드 v_p, v_q 의 유사도를 아래 식으로 계산하여 간선에 부여
 - $f(v)$ 는 v 에 해당하는 픽셀의 색상(R, G, B)과 위치 (x, y) 를 결합한 5차원 벡터
 - v 가 슈퍼 픽셀인 경우 슈퍼픽셀 내 픽셀들의 평균을 사용

$$\text{거리} \begin{cases} d_{pq} = \|f(v_p) - f(v_q)\|, \text{ 만일 } v_q \in \text{neighbor}(v_p) \\ \infty, \text{ 그렇지 않으면} \end{cases} \quad D: \text{상수}$$

$$\text{유사도} \begin{cases} s_{pq} = D - d_{pq} \text{ 또는 } \frac{1}{e^{d_{pq}}}, \text{ 만일 } v_q \in \text{neighbor}(v_p) \\ 0, \text{ 그렇지 않으면} \end{cases}$$



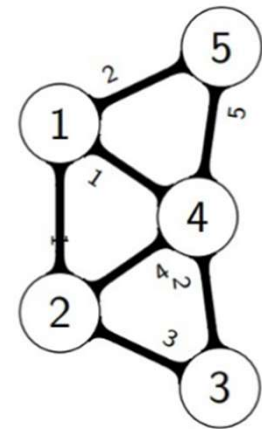
최적화 분할

- 영상의 그래프 표현
 - Normalized cut(N-cut, 정규화 절단)알고리즘
 - cut은 영상을 두 영역으로 분할했을 때 분할의 좋은 정도를 측정해 주는 목적 함수
 - C_1 과 C_2 가 클수록 둘 사이에 간선이 많아 cut은 덩달아 커지므로 cut을 사용한 분할 알고리즘은 영역을 자잘하게 분할하는 경향이 있음

$$cut(C_1, C_2) = \sum_{v_p \in C_1, v_q \in C_2} s_{pq}$$

- N-cut은 cut을 정규화하여 영역의 크기에 중립이 되게 해줌

$$ncut(C_1, C_2) = \frac{cut(C_1, C_2)}{cut(C_1, C)} + \frac{cut(C_2, C)}{cut(C_2, C)}$$



최적화 분할

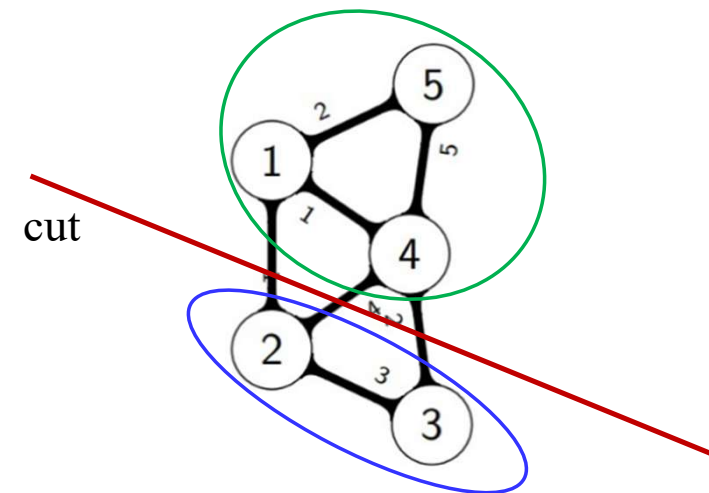
- 영상의 그래프 표현
 - Normalized cut(N-cut, 정규화 절단)알고리즘
 - cut은 영상을 두 영역으로 분할했을 때 분할의 좋은 정도를 측정해 주는 목적 함수
 - C_1 과 C_2 가 클수록 둘 사이에 간선이 많아 cut은 덩달아 커지므로 cut을 사용한 분할 알고리즘은 영역을 자잘하게 분할하는 경향이 있음

$$cut(C_1, C_2) = \sum_{v_p \in C_1, v_q \in C_2} s_{pq}$$

- N-cut은 cut을 정규화하여 영역의 크기에 중립이 되게 해줌

$$ncut(C_1, C_2) = \frac{cut(C_1, C_2)}{\boxed{cut(C_1, C)}} + \frac{cut(C_1, C_2)}{cut(C_2, C)}$$

C_1 과 C 가 얼마나 강하게
연결되어 있는가?



최적화 분할

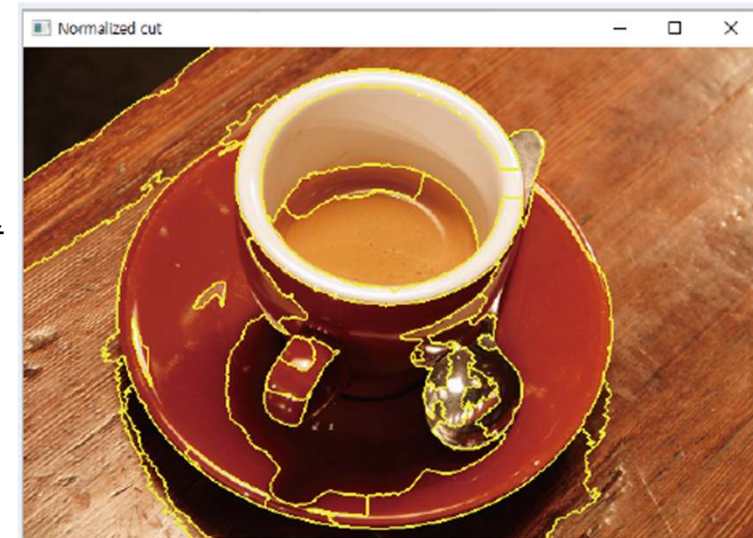
- 영상의 그래프 표현
 - Normalized cut(N-cut, 정규화 절단)알고리즘
 - cut은 영상을 두 영역으로 분할했을 때 분할의 좋은 정도를 측정해 주는 목적 함수
 - C_1 과 C_2 가 클수록 둘 사이에 간선이 많아 cut은 덩달아 커지므로 cut을 사용한 분할 알고리즘은 영역을 자잘하게 분할하는 경향이 있음

$$cut(C_1, C_2) = \sum_{v_p \in C_1, v_q \in C_2} s_{pq}$$

- N-cut은 cut을 정규화하여 영역의 크기에 중립이 되게 해줌

$$ncut(C_1, C_2) = \frac{cut(C_1, C_2)}{cut(C_1, C)} + \frac{cut(C_2, C)}{cut(C_2, C)}$$

- 한계
 - 색상과 거리 정보에만 의존하므로 의미 분할 불가능



최적화 분할 실습

```
import urllib.request

import matplotlib.pyplot as plt

from skimage.segmentation import slic, mark_boundaries
from skimage.color import label2rgb
from skimage import graph # N-Cut 계산을 위한 그래프 모듈

# 샘플 이미지 로드

url =
'https://raw.githubusercontent.com/opencv/opencv/master/samples/data/butterfly.jpg'
urllib.request.urlretrieve(url, 'butterfly.jpg')
img = cv2.imread('butterfly.jpg')
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

# 2. SLIC 슈퍼픽셀 분할 (Oversegmentation 상태, 400조각)
labels = slic(img_rgb, n_segments=400, compactness=10, sigma=1, start_label=1)
```

슈퍼픽셀들 간의 인접 관계와 색상 차이를 계산하여 그래프 생성

```
g = graph.rag_mean_color(img_rgb, labels)
```

N-Cut (Normalized Cut) 적용

thresh: 잘라낼 기준값. 낮을수록 더 많이 쪼개지고, 높을수록 크게 뭉침

num_cuts: 반복해서 자를 횟수

```
nc_labels = graph.cut_normalized(labels, g, thresh=1.0, num_cuts=2)
```

결과 시각화 준비

```
slic_avg = label2rgb(labels, img_rgb, kind='avg') # SLIC 결과
```

```
ncut_avg = label2rgb(nc_labels, img_rgb, kind='avg') # N-Cut 결과
```

최적화 분할 실습

```
# 최종 출력
```

```
plt.figure(figsize=(20, 5))
```

```
plt.subplot(1, 4, 1)
```

```
plt.title("1. Original")
```

```
plt.imshow(img_rgb)
```

```
plt.axis('off')
```

```
plt.subplot(1, 4, 2)
```

```
plt.title("2. SLIC Boundaries")
```

```
plt.imshow(mark_boundaries(img_rgb, labels))
```

```
plt.axis('off')
```

```
plt.subplot(1, 4, 3)
```

```
plt.title("3. SLIC Avg Color")
```

```
plt.imshow(slic_avg.astype('uint8'))
```

```
plt.axis('off')
```

```
plt.subplot(1, 4, 4)
```

```
plt.title("4. N-Cut Result")
```

```
plt.imshow(ncut_avg.astype('uint8'))
```

```
plt.axis('off')
```

```
plt.tight_layout()
```

```
plt.show()
```


영상의 품질 측정 지표

- PSNR(Peak Signal-to-Noise Ratio, 최대 신호 대 잡음비)
 - 신호(Signal, 원본영상) 대비 잡음(Noise)이 얼마나 섞였는가를 수치로 나타낸 것

$$PSNR = 10 \cdot \log_{10} \left(\frac{MAX_I^2}{MSE} \right) \text{ dB(데시벨)}$$

- MAX_I : 해당 영상의 픽셀이 가질 수 있는 최대값
(일반적인 8비트 영상에서는 255)
 - MSE : 원본과 결과 영상의 픽셀 값 차이를 제공하여 평균 낸 값
- 값이 클수록 좋은 화질(원본에 더 가까운 화질)
 - 약 30dB 이상이면 보통 좋은 화질로 평가
 - 인간의 시각 시스템 특성을 완전히 반영하지 못한다는 단점이 있음

영상의 품질 측정 지표

- PSNR(Peak Signal-to-Noise Ratio, 최대 신호 대 잡음비)
 - 신호(Signal, 원본영상) 대비 잡음(Noise)이 얼마나 섞였는가를 수치로 나타낸 것

$$PSNR = 10 \cdot \log_{10} \left(\frac{MAX_I^2}{MSE} \right) \text{ dB(데시벨)}$$

- MAX_I : 해당 영상의 픽셀이 가질 수 있는 최대값
(일반적인 8비트 영상에서는 255)
- MSE : 원본과 결과 영상의 픽셀 값 차이를 제공하여 평균 낸 값

JPEG



Original image

PSNR 34.8227dB

PSNR 30.9394dB

PSNR 25.8699dB

JPEG : 가장 많이 쓰이는 (정지)영상의 손실압축기술

영상의 품질 측정 지표

- SSIM(Structural Similarity Index Map, 구조적 유사도)
 - 수치적인 차이보다 사람의 눈에 얼마나 비슷하게 보이는가에 초점을 맞춘 영상 품질 측정 지표
 - PSNR의 한계 : PSNR은 단순히 픽셀의 값이 얼마나 달라졌는지만 측정. 그러나 우리 눈은 물체의 경계가 뭉개지거나 노이즈가 끼는 것에 훨씬 민감
 - SSIM은 원본 영상(x)과 왜곡된 영상(y)을 비교할 때 다음 세 가지 지표를 결합하여 계산

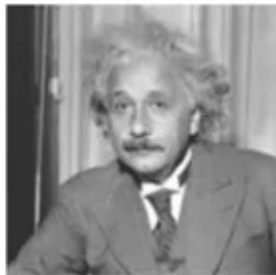
$$\text{SSIM}(x, y) = l(x, y)^\alpha \cdot c(x, y)^\beta \cdot s(x, y)^\gamma \quad \text{보통 } \alpha = \beta = \gamma = 1$$

- 휘도 (l , Luminance): 평균 밝기가 얼마나 변했는가?
 - 대비 (c , Contrast): 밝기값의 표준편차가 얼마나 변했는가?
 - 구조 (s , Structure): 두 영상의 픽셀 간 상관관계가 유지되는가? (물체의 형태나 패턴이 깨졌는가?)
- SSIM은 밝기가 변해도 물체의 형태(구조)가 유지되면 높은 점수를 줌. 즉, 사람의 주관적 화질 평가와 더 일치
- 약 0.9 이상이면 우수한 화질로 평가

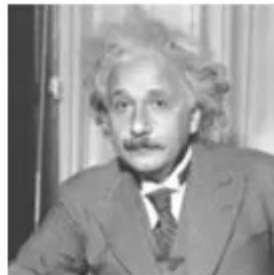
영상의 품질 측정 지표

- SSIM(Structural Similarity Index Map, 구조적 유사도)
 - 수치적인 차이보다 사람의 눈에 얼마나 비슷하게 보이는가에 초점을 맞춘 영상 품질 측정 지표
 - PSNR의 한계 : PSNR은 단순히 픽셀의 값이 얼마나 달라졌는지만 측정. 그러나 우리 눈은 물체의 경계가 뭉개지거나 노이즈가 끼는 것에 훨씬 민감
 - SSIM은 원본 영상(x)과 왜곡된 영상(y)을 비교할 때 다음 세 가지 지표를 결합하여 계산

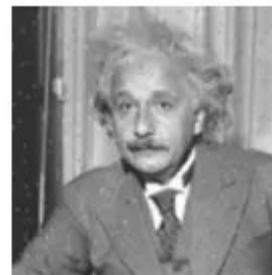
$$\text{SSIM}(x, y) = l(x, y)^\alpha \cdot c(x, y)^\beta \cdot s(x, y)^\gamma \quad \text{보통 } \alpha = \beta = \gamma = 1$$



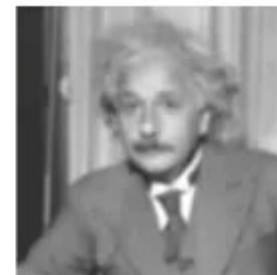
Original
SSIM=1



PSNR=26.547
SSIM=0.988



PSNR=26.547
SSIM=0.840



PSNR=26.547
SSIM=0.694

영상의 품질 측정 실습

```
import urllib.request

from skimage.metrics import structural_similarity as ssim
from google.colab.patches import cv2_imshow

# 샘플 이미지 다운로드

url = '
https://raw.githubusercontent.com/opencv/opencv/master/samples/data/baboon.jpg '

Urllib.request.urlretrieve(url, ' baboon.jpg ' )

Original = cv2.imread( ' baboon.jpg ' )

# 영상 축소 및 재확대 (보간법 적용)

height, width = original.shape[:2]

# 1/4 크기로 축소 (INTER_AREA)

small = cv2.resize(original, (width//4, height//4), interpolation=cv2.INTER_AREA)

# 다시 원래 크기로 확대 (INTER_LINEAR )

# INTER_NEAREST, INTER_CUBIC로도 실습

restored = cv2.resize(small, (width, height), interpolation=cv2.INTER_LINEAR)
```

```
# PSNR 계산 (OpenCV 내장 함수)

psnr_val = cv2.PSNR(original, restored)

# SSIM 계산 (scikit-image 사용)

ssim_val = ssim(original, restored, channel_axis=2)

# 4. 결과 출력

print(f"--- 품질 측정 결과 ---")

print(f"PSNR: {psnr_val:.2f} dB")

print(f"SSIM: {ssim_val:.4f}")

print("-----")

# 가로로 붙여서 확인

combined = np.hstack((original, restored))

print("\n[왼쪽: 원본 | 오른쪽: 1/4 축소 후 복원본]")

cv2_imshow(cv2.resize(combined, (0,0), fx=0.8, fy=0.8)) # 화면 크기에 맞춰 조절
```

내용 정리

- 영상 처리 알고리즘 실습
- 에지 및 코너 탐지
- 영상의 품질 측정 지표

다음 주 강의 내용

- 영상의 변환(Transform) 및 필터링
- 영상의 특징 검출 및 매칭 기술(SIFT 등)